

Università di Verona
Dipartimento di Informatica

Corso di Sistemi Operativi
Progetto di Laboratorio – Anno accademico 2024/2025

Relazione di progetto

(Servizio di calcolo dell'impronta SHA-256 di file multipli)

Opzione 1 – IPC System V su Linux / macOS

Studente: Angelo Vaccaro
Matricola: VR480616
Corso di laurea: Informatica

Indice

1	Introduzione	2
1.1	Requisiti e funzionalità realizzati	2
1.2	Ambiente e compilazione	2
2	Architettura	3
2.1	Componenti	3
2.2	Risorse IPC	3
2.3	Flusso di una richiesta	4
3	Comunicazione tra i processi	5
3.1	Code di messaggi	5
3.2	Trasferimento del file tramite memoria condivisa	5
3.3	Sincronizzazione con i semafori	5
4	Gestione delle richieste	6
4.1	Elaborazione concorrente con processi distinti	6
4.2	Limite al numero di processi	6
4.3	Schedulazione delle richieste pendenti	6
4.4	Client di amministrazione e stato del server	7
5	Affidabilità e gestione degli errori	7
5.1	Comunicazione del motivo al client	8
5.2	Rilevamento della morte del server	8
5.3	Sveglia periodica del server	9
6	Problematiche affrontate e scelte progettuali	9
7	Verifica sperimentale	10
8	Limitazioni e possibili estensioni	12
9	Organizzazione del codice	12

1 Introduzione

Il progetto consiste in un servizio che calcola l'impronta SHA-256 di file, realizzato con i meccanismi di intercomunicazione System V (code di messaggi, memoria condivisa, semafori). Il sistema è composto da tre programmi: un *server*, che accoglie le richieste e ne coordina l'esecuzione; un *client*, che invia un file al server e riceve l'impronta calcolata; un client di amministrazione, *admin*, che permette di modificare i parametri del servizio durante l'esecuzione e di interrogarne lo stato.

1.1 Requisiti e funzionalità realizzati

La tabella 1 riporta i requisiti dell'opzione 1 e le funzionalità aggiuntive realizzate, con le sezioni in cui sono descritti.

Requisito	Realizzazione	Sez.
Server e client comunicano tramite code di messaggi	coda unica, messaggi tipizzati	3.1
Il file è trasferito da client a server tramite memoria condivisa	segmento per richiesta, a blocchi	3.2
Processi distinti per le richieste concorrenti	un processo figlio per richiesta	4.1
Limite al numero di processi, modificabile da un secondo client	comando <code>admin limit</code>	4.2
Schedulazione delle richieste pendenti per dimensione del file	politica SJF	4.3
Semafori per sincronizzare il flusso di comunicazione	coppia di semafori per richiesta	3.3
<i>Funzionalità aggiuntive</i>		
Più algoritmi di schedulazione, scelti all'avvio del server	<code>sjf</code> e <code>fcfs</code>	4.3
Interrogazione dello stato corrente del server	comando <code>admin status</code>	4.4

Tabella 1: Requisiti dell'opzione 1, funzionalità aggiuntive e loro realizzazione.

1.2 Ambiente e compilazione

Il codice è scritto in C11 ed è organizzato come progetto CMake. Lo sviluppo e la verifica sono stati condotti su Linux (Ubuntu 26.04 in WSL2, kernel 6.18) con gcc 15.2, CMake 4.2 e OpenSSL 3.5. Il codice si limita a chiamate System V e POSIX disponibili anche su macOS, ma su tale piattaforma non è stato collaudato (sezione 8).

```
cmake -S . -B build && cmake --build build
build/server -p 2 -s sjf          # avvio del server
build/client file.bin             # stampa "<impronta> file.bin"
build/admin limit 4               # modifica del limite di processi
build/admin status                # richieste in esecuzione e in attesa
```

Il server accetta le opzioni `-p` (numero massimo di processi, predefinito 2), `-s` (politica di schedulazione: `sjf`, predefinita, oppure `fcfs`) e `-d` (ritardo artificiale per blocco, in millisecondi, impiegato nelle prove per rendere osservabili le code).

Nell'implementazione si adottano le convenzioni del corso: chiave della coda ottenuta con `ftok`, permessi `S_IRUSR | S_IWUSR`, gestione degli errori con la funzione `errExit`, inizializzazione dei semafori con `union semun` e `SETALL`, operazioni sui semafori con `semop` e `struct sembuf`, segnali con `signal` e `alarm`, creazione dei processi con `switch (fork())`.

2 Architettura

2.1 Componenti

server Processo padre che riceve i messaggi, mantiene la coda delle richieste pendenti e decide quando avviare un processo *worker* per ciascuna. Non calcola impronte e non accede ai file.

worker Processo figlio del server, creato con `fork`, che serve una singola richiesta: legge il file dalla memoria condivisa, calcola l'impronta, risponde al client e termina.

client Legge il file e lo deposita, un blocco alla volta, nella memoria condivisa; attende quindi l'impronta.

admin Client di amministrazione: modifica il limite di processi e interroga lo stato del server.

2.2 Risorse IPC

Coda di messaggi. Il sistema usa un'unica coda, la cui chiave è ricavata con `ftok` (percorso `/tmp`, identificativo `'S'`). La coda è creata dal server con `IPC_CREAT | IPC_EXCL`, in modo che un secondo server sulla stessa coda sia rifiutato; client e admin la aprono soltanto, e la sua assenza indica che il server non è in esecuzione.

I messaggi sono distinti dal campo `mtype` (tabella 2). I tipi più bassi sono destinati al server; le risposte usano `MSG_REPLY_BASE + pid` del destinatario, così che ogni processo riceva solo i messaggi a lui indirizzati pur condividendo la coda.

<code>mtype</code>	Da → a	Contenuto
1 (<code>MSG_DONE</code>)	worker → server	termine dell'elaborazione: lo slot può essere liberato
2 (<code>MSG_ADMIN</code>)	admin → server	comando: nuovo limite oppure richiesta di stato
3 (<code>MSG_REQUEST</code>)	client → server	nome e dimensione del file, identificativi di memoria condivisa e semafori
1000 + pid	server/worker → processo	risposta: impronta, esito o testo di stato

Tabella 2: Tipi di messaggio.

Il server riceve con `msgtyp = -3`: viene estratto per primo il messaggio con il tipo più basso, per cui le notifiche di fine elaborazione e i comandi di amministrazione precedono le nuove richieste. Il messaggio è una struttura a dimensione fissa di 128 byte, con un campo di testo di 80 byte, sufficiente per un'impronta esadecimale (64 caratteri più il terminatore); un `_Static_assert` verifica in compilazione che tale dimensione non scenda sotto questa soglia.

Memoria condivisa e semafori. Ogni client crea, con `IPC_PRIVATE`, un segmento di memoria condivisa (4100 byte: un contatore `len` e un blocco dati da 4096 byte) e un insieme di due semafori, e ne comunica gli identificativi al server nel messaggio di richiesta. L'uso di risorse distinte per ogni richiesta rende indipendenti i trasferimenti concorrenti.

Semaforo di presenza del server. Un insieme di un solo semaforo, con la stessa chiave della coda (`msgget` e `semget` hanno spazi di identificatori separati, quindi non c'è conflitto), segnala se il server è vivo. Il server lo porta a 1 con `SEM_UNDO` all'avvio: se termina in qualunque modo, anche con `kill -9`, il kernel lo riporta a 0 da solo, senza che il server debba fare nulla per questo. Il client lo interroga per distinguere un server occupato, che va semplicemente atteso, da uno morto senza pulire (sezione 5.2), caso in cui la coda di messaggi, a differenza di questo semaforo, non si rimuove da sola.

2.3 Flusso di una richiesta

La figura 1 illustra il ciclo di vita di una richiesta.

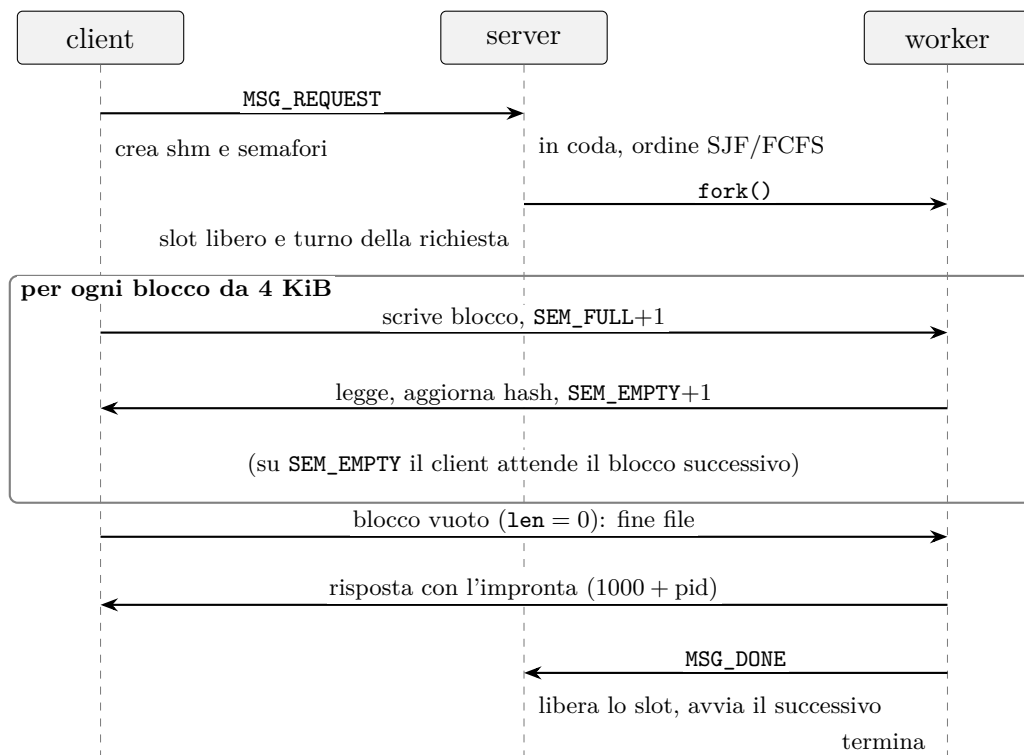


Figura 1: Ciclo di vita di una richiesta tra client, server e worker.

1. Il client crea memoria condivisa e semafori e invia `MSG_REQUEST`, indicando anche la dimensione del file, su cui il server basa la schedulazione.
2. Il server inserisce la richiesta nell'elenco delle pendenti. Se il numero di worker in esecuzione è inferiore al limite, seleziona la prossima richiesta secondo la politica in uso e crea un worker con `fork`.
3. Client e worker trasferiscono il file blocco per blocco attraverso la memoria condivisa, sincronizzati dai semafori (sezione 3.3).

4. Il worker risponde direttamente al client con l'impronta e notifica il server con `MSG_DONE`; il server libera lo slot e, se vi sono richieste in attesa, ne avvia un'altra.
5. Il client, ricevuta la risposta, rimuove memoria condivisa e semafori.

Il server non apre mai il file: il nome contenuto nel messaggio serve unicamente per i log e per la visualizzazione dello stato, mentre il contenuto giunge esclusivamente tramite la memoria condivisa. Ciò è stato verificato con `strace`.

3 Comunicazione tra i processi

3.1 Code di messaggi

Richieste e risposte transitano esclusivamente dalla coda descritta nella sezione precedente. Dopo aver trasferito il file, il client attende con `msgrcv` un messaggio di tipo `MSG_REPLY_BASE + pid` e stampa l'impronta nel formato di `sha256sum` (impronta, due spazi, nome del file). Un file inesistente o un server non attivo sono segnalati con un messaggio d'errore e un codice di uscita diverso da zero.

3.2 Trasferimento del file tramite memoria condivisa

Il segmento condiviso costituisce un buffer a un solo posto, riempito dal client e svuotato dal worker. Contiene il numero di byte validi (`len`) e il blocco dati. Un valore di `len` pari a zero indica la fine del file; un valore riservato (`CHUNK_ABORT`) segnala un errore di lettura nel client e l'abbandono del trasferimento.

I blocchi sono da 4 KiB. Il worker aggiorna l'impronta con `SHA256_Update` su porzioni da 32 byte, come nel codice di esempio fornito con le specifiche, così che il tempo di calcolo resti significativo anche per file di dimensioni modeste.

3.3 Sincronizzazione con i semafori

Il buffer condiviso è protetto da un insieme di due semafori per richiesta, secondo lo schema produttore/consumatore (tabella 3).

Semaforo	Valore iniziale	Attesa (P)	Segnalazione (V)
<code>SEM_EMPTY</code>	1 (buffer libero)	client, prima di scrivere	worker, dopo aver letto
<code>SEM_FULL</code>	0 (buffer vuoto)	worker, prima di leggere	client, dopo aver scritto

Tabella 3: Semafori associati a ciascuna richiesta.

I valori iniziali sono impostati con `semctl(..., SETALL, ...)`; le operazioni P e V sono eseguite con `semop`, incapsulata nella funzione `semOp`; il client ne usa una variante, `semOpClient`, che gestisce anche la rimozione dei semafori da parte del server (sezione 5.1) e un controllo periodico che il server sia ancora vivo (sezione 5.2). Il nucleo del protocollo è il seguente.

```

/* client */
do {
    semOp(semid, SEM_EMPTY, -1);
    bR = read(file, buf->data, CHUNK);
/* worker */
while (1) {
    semOp(semid, SEM_FULL, -1);
    len = buf->len;

```

```

    buf->len = bR;
    semOp(semid, SEM_FULL, +1);
} while (bR > 0);

    if (len == 0) break; /* fine */
    SHA256_Update(...);
    semOp(semid, SEM_EMPTY, +1);
}

```

Il client deposita subito il primo blocco, anche se il server non ha ancora avviato il worker; per il secondo attende `SEM_EMPTY`, che il worker segnala solo dopo aver letto il primo. Il semaforo sincronizza così anche la fase di attesa in coda con l'inizio dell'elaborazione, senza che il client debba conoscere lo stato del server.

4 Gestione delle richieste

4.1 Elaborazione concorrente con processi distinti

Per ogni richiesta avviata il server crea un processo figlio con la struttura `switch (fork())` (caso -1: errore; caso 0: figlio; caso di default: padre). I figli terminano autonomamente con `exit` e il padre li raccoglie con `waitpid` non bloccante. Subito dopo la `fork`, il figlio riporta i gestori di `SIGINT`, `SIGTERM` e `SIGALRM` all'azione predefinita, per non ereditare quelli del padre.

4.2 Limite al numero di processi

Il server conserva il numero di worker in esecuzione e non ne avvia altri una volta raggiunto il limite, fissato all'avvio con `-p` (predefinito 2, massimo 64). Il limite può essere modificato durante l'esecuzione con `admin limit N`, che invia un messaggio `MSG_ADMIN`:

- se il limite aumenta, le richieste in attesa vengono avviate fino al nuovo limite;
- se scende sotto il numero di worker attivi, questi non vengono interrotti: non ne viene avviato alcun altro finché il loro numero non è inferiore al limite;
- i valori non validi (zero, negativi, superiori a 64, non numerici) sono rifiutati con un messaggio.

4.3 Schedulazione delle richieste pendenti

Le richieste in attesa sono conservate in un array in ordine di arrivo. Quando si libera uno slot il server estrae la prossima richiesta secondo la politica scelta all'avvio con `-s`:

sjf (predefinita) sceglie la richiesta con il file più piccolo; a parità di dimensione, la più vecchia. È lo *Shortest Job First* applicato alla dimensione del file, che è proporzionale al tempo di calcolo: la politica tende a minimizzare il tempo medio di attesa. La dimensione è nota perché il client la comunica nella richiesta.

fcfs serve le richieste nell'ordine di arrivo.

Le due politiche differiscono solo nella funzione che estrae la richiesta dall'array. A titolo di esempio, con limite 1 e richieste di 3 MB, 800 KB, 10 B, 400 KB e 300 KB giunte in quest'ordine, dopo la prima la sequenza di avvio è 10 B, 300 KB, 400 KB, 800 KB con **sjf** e 800 KB, 10 B, 400 KB, 300 KB con **fcfs**.

Applicata da sola, la politica **sjf** esporrebbe un file di grandi dimensioni al rischio di un'attesa indefinita, se continuano ad arrivare file più piccoli (*starvation*). Per questo alla dimensione si applica un *aging*: la dimensione usata per la scelta non è quella effettiva del file ma una dimensione ridotta in base al tempo di attesa in coda,

$$dimensione_invecchiata = \max(0, dimensione - TASSO \times attesa),$$

con $TASSO = 10\,000$ byte per ogni secondo di attesa. Una richiesta ferma in coda vede quindi la propria dimensione scendere progressivamente, finché non diventa la più piccola e viene scelta indipendentemente dalla dimensione reale del file. L'aging si applica solo alla politica **sjf**: la **fcfs** è per costruzione priva di starvation, poiché serve sempre la richiesta più vecchia.

4.4 Client di amministrazione e stato del server

Il programma **admin** offre due comandi. **admin limit N**, già descritto, modifica il limite di processi. **admin status** interroga il server, che risponde con la politica in uso, il limite, il numero di worker attivi, le richieste in esecuzione (con il pid del worker e del client) e quelle in attesa, in ordine di arrivo.

```
policy: sjf | limit: 1 | running: 1 | pending: 2
RUNNING:
  worker 6921 <- client 6919 grande (3000000 B)
PENDING (arrival order):
  client 6922 medio (300000 B)
  client 6923 piccolo (10 B)
```

La risposta è composta da più messaggi, uno per riga, con un campo **more** che indica se ne seguono altri: ogni messaggio resta così piccolo e di dimensione fissa. Le righe per sezione sono limitate a 8; le eccedenti sono riassunte da una riga finale "... and N more". Per non attendere indefinitamente, **admin** termina se il server non risponde entro 5 secondi (**alarm**).

5 Affidabilità e gestione degli errori

Le risorse System V sono persistenti: non vengono rilasciate alla terminazione del processo che le ha create. Per questo il sistema gestisce esplicitamente le terminazioni anomale (tabella 4).

Evento	Comportamento
Client interrotto (Ctrl-C, SIGTERM)	il gestore di segnale rimuove memoria condivisa e semafori prima di uscire
Client ucciso con <code>kill -9</code> mentre è in coda	il server verifica, a ogni ciclo (almeno una volta al secondo), che i client in coda esistano, con <code>kill(pid, 0)</code> ; se uno è scomparso, scarta la richiesta e rimuove le sue risorse
Client ucciso durante il trasferimento	il worker attende ogni blocco con un <code>alarm(10)</code> ; allo scadere termina, e il server libera lo slot, avvisa il client e rimuove le risorse
Worker terminato in modo anomalo	il server, raccogliendo il figlio senza aver ricevuto <code>MSG_DONE</code> , risponde con un errore al client e rimuove semafori e memoria
Arresto del server (SIGINT, SIGTERM)	termina i worker, avvisa i client delle richieste attive e pendenti, ne rimuove i semafori (sbloccandoli) e infine la coda
Più di 100 richieste in attesa	la richiesta è rifiutata con un messaggio d'errore e le sue risorse sono rimosse
Coda già esistente all'avvio	il server non parte e indica il comando <code>ipcrm -Q</code> per rimuovere un eventuale residuo
Coda piena	il server invia con <code>IPC_NOWAIT</code> , per non bloccarsi; worker e client inviano in modo bloccante
Server ucciso con <code>kill -9</code> (client già in attesa o connesso dopo)	il client rileva l'assenza del server tramite il semaforo di presenza ed esce con un messaggio chiaro entro pochi secondi, invece di restare bloccato

Tabella 4: Eventi anomali gestiti.

5.1 Comunicazione del motivo al client

Quando il server interrompe una richiesta (rifiuto, worker terminato in modo anomalo, arresto), rimuove i semafori del client per sbloccarlo. Prima di farlo invia però al client un messaggio di risposta con il motivo. Il client, se un'operazione su un semaforo fallisce con `EIDRM` o con `EINVAL` (quest'ultimo osservato sotto carico molto elevato, quando la rimozione avviene mentre il client sta per chiamare `semop` e non ancora in attesa), legge tale messaggio senza attendere (`IPC_NOWAIT`) e lo mostra. Se la coda era troppo piena per contenere la risposta, il client segnala comunque che la richiesta è stata interrotta dal server.

5.2 Rilevamento della morte del server

Le risorse di una singola richiesta (memoria condivisa, semafori) permettono al client di accorgersi che il server le ha rimosse, ma questo non basta se il server muore prima ancora di riceverla, o se muore in un modo che non rimuove nulla (`kill -9`): la coda di messaggi resta, indistinguibile da quella di un server regolarmente attivo. Per questo il client non attende mai un semaforo di richiesta senza limite: ogni `SERVER_CHECK` (2) secondi un `alarm` interrompe `semop` con `EINTR`, e il client controlla il valore del semaforo di presenza (sezione 2.2). Se è ancora 1 il server è soltanto occupato, e l'attesa riprende senza limiti di durata, anche per molti secondi, senza falsi allarmi; se è 0 il client stampa un messaggio chiaro ed esce, invece di restare bloccato indefinitamente.

5.3 Sveglia periodica del server

Il server padre è per lo più bloccato in `msgrcv`. Per rilevare client scomparsi e worker terminati in modo anomalo indipendentemente dall'arrivo di messaggi, arma una `alarm(1)` che si riarma a ogni scatto: ogni secondo `SIGALRM` interrompe `msgrcv`, che ritorna con `EINTR`, e il ciclo principale ricontrolla il flag di arresto ed esegue le verifiche. I gestori sono installati con `signal; msgrcv` non viene mai riavviata automaticamente dopo un segnale, comportamento qui necessario.

6 Problematiche affrontate e scelte progettuali

Capacità limitata della coda. Una coda System V ha una capienza complessiva in byte (16 KiB per impostazione predefinita su Linux), non solo un numero di messaggi. Con messaggi da 4 KiB ne entrerebbero quattro. I messaggi sono quindi piccoli e di dimensione fissa (128 byte, circa 130 per coda) e i blocchi del file transitano dalla memoria condivisa anziché dai messaggi.

Rischio di stallo sulla coda. La coda è condivisa: se fosse piena di richieste e il server tentasse di inviare una risposta con una `msgsnd` bloccante, resterebbe in attesa di uno spazio che solo esso, leggendo, può liberare. Per questo tutti gli invii del server usano `IPC_NOWAIT`. Worker e client possono invece bloccarsi, poiché il server continua a svuotare la coda.

Notifica di fine elaborazione. La sola segnalazione `SIGCHLD` non basta a informare il server della terminazione di un worker: un segnale che giunge subito prima di `msgrcv` non la interrompe e andrebbe perso. Il worker invia quindi un messaggio `MSG_DONE` sulla stessa coda, che sveglia il server con certezza; la sveglia periodica copre il caso residuo di un worker che muore senza inviarlo.

Race condition fra la terminazione del worker e la lettura del suo `MSG_DONE`. La funzione che raccoglie i worker terminati usava `waitpid` da sola: un worker diventa “zombie”, e quindi visibile a `waitpid`, solo *dopo* aver già completato il proprio `msgsnd(MSG_DONE)`, ma se quel messaggio non era ancora stato letto dal ciclo principale (che ne legge uno alla volta, insieme ai messaggi di altro tipo), veniva scambiato per un worker terminato senza notifica. Il server inviava allora al client una risposta duplicata che nessuno leggeva più, dato che il client aveva già ricevuto quella corretta dal worker stesso, e che restava quindi in coda. Con 105 client avviati simultaneamente (otto worker, nessun ritardo artificiale) si osservavano così 4 eventi spuri; ripetendo lo stesso burst più volte su un server già carico (un piccolo ritardo artificiale, per rendere l'effetto osservabile) il fenomeno si accumulava, fino a saturare la coda – 135 messaggi, a fronte di una capienza di circa 128 – entro un migliaio di richieste consecutive. In quella condizione, interrogazioni concorrenti di `admin status` fallivano sistematicamente, terminando dopo 5 secondi per il proprio `alarm`: una risposta multi-messaggio perdeva silenziosamente uno dei suoi pezzi quando la coda non aveva più spazio, perché `sendReply` invia con `IPC_NOWAIT` senza controllare l'esito di `msgsnd`. La correzione raccoglie prima *tutti* i pid terminati con `waitpid`, poi svuota i `MSG_DONE` eventualmente arrivati nel frattempo, e solo a questo punto un pid ancora segnato come attivo è davvero terminato senza notifica; invertire l'ordine (svuotare e poi chiamare `waitpid`) lascia invece una finestra residua, osservata empiricamente in circa un caso ogni mille richieste. Dopo la correzione, su tre ripetizioni indipendenti di 4200 richieste ciascuna (12600 in totale) non si è più verificato alcun evento spurio, la coda è rimasta vuota, e trenta interrogazioni di `admin status` durante un nuovo carico concorrente sono andate tutte a buon fine (tabella 5).

Il motivo dell'arresto non sempre recapitato. Alla chiusura, il server rimuove i semafori dei client attivi e pendenti, sbloccandoli, e poi la coda di messaggi. Un client appena sbloccato deve però fare in tempo a leggere, con `msgrcv(IPC_NOWAIT)`, la risposta specifica che il server gli aveva già inserito in coda, prima che la coda stessa venga rimossa: se il sistema operativo non lo schedula abbastanza in fretta, quella lettura fallisce e il client mostra il messaggio generico invece del motivo. Misurato su 15 arresti ripetuti con più client in attesa (143 interruzioni osservate), il motivo specifico arrivava solo nel 64% dei casi. La correzione introduce un margine di grazia, fino a mezzo secondo e interrotto non appena la coda risulta vuota, fra lo sblocco dei client e la rimozione della coda: sulla stessa prova, ripetuta due volte (300 interruzioni), il motivo è arrivato sempre, senza rallentare in modo percepibile un arresto senza client in attesa (circa 5 ms).

Client che scompaiono durante il trasferimento. Se il client termina a metà trasferimento, il worker resterebbe in attesa di un blocco che non arriverà, occupando uno slot: con limite 1 l'intero servizio si bloccherebbe. Il problema è stato individuato provando l'uccisione del client con `kill -9`. La soluzione adottata è la sveglia del worker, che con l'azione predefinita di `SIGALRM` lo termina; il server riconosce la terminazione anomala e ripulisce.

Proprietà delle risorse. Memoria condivisa e semafori sono creati e rimossi dal client, che ne conosce la durata. Il server interviene solo quando il client non può più farlo (terminazione anomala o arresto del server stesso). Due percorsi che rifiutano una richiesta appena arrivata (coda dei pendenti piena, `fork` fallita) rimuovevano però solo il semaforo e non la memoria condivisa: un'asimmetria innocua se il client è vivo, perché la rimuove lui stesso, ma che lascia un segmento orfano se è già morto in quell'istante. È stata corretta rendendo simmetrica anche questa rimozione, come già avveniva negli altri percorsi di errore.

Race condition sulla rimozione dei semafori. Verificando il limite di 100 richieste pendenti con 105 client avviati simultaneamente è emerso che `semop` può fallire con `EINVAL` anziché con `EIDRM` quando il server rimuove l'insieme di semafori proprio nell'istante in cui il client sta per invocarla. Il client trattava solo `EIDRM` come segnale di richiesta interrotta, e terminava con un messaggio di sistema invece di quello del server. La correzione è stata verificata ripetendo la stessa prova e con uno stress test più intenso, a 150 client (tabella 5), che tratta allo stesso modo entrambi i codici di errore.

7 Verifica sperimentale

Il sistema è stato collaudato con una suite automatica (`tests/run_tests.sh`, 23 controlli) e con prove manuali, riassunte nella tabella 5. Le prove sono state ripetute a partire da una compilazione pulita e a ogni ripetizione si è verificato con `ipcs` l'assenza di risorse IPC residue.

Tabella 5: Prove eseguite ed esiti.

Prova	Esito
Impronta di file vuoto, di 10 B, di 4096 B, di 300 KB, di 20 MB	identica a <code>sha256sum</code>
40 client concorrenti sullo stesso file, limite 4	40 risultati corretti

continua nella pagina successiva

Tabella 5 (continua)

Prova	Esito
Il server apre il file? (strace)	mai
Segmento condiviso e semafori durante il trasferimento (ipcs)	presenti; 2 processi agganciati al segmento
6 richieste con limite 3	3 processi figli distinti
Limite abbassato / alzato da admin a runtime	nessun nuovo avvio / avvio immediato dei pendenti
Limiti non validi (0, 65, testo)	rifiutati
Ordine di avvio con sjf e con fcfs	crescente per dimensione / d'arrivo
Richiesta di 25 KB in coda durante l'elaborazione di un file da 1,2 MB, con 21 richieste da 10 B arrivate nel frattempo ogni 0,2 s	la richiesta da 25 KB viene avviata subito dopo, prima di tutte le richieste più piccole appena arrivate
admin status con nomi lunghi e molte richieste	righe troncate, riga finale "and N more"
Più di 8 worker attivi contemporaneamente	mostrate le prime 8, poi "and N more", come per le richieste in attesa
Oltre 100 richieste in attesa	le eccedenti sono rifiutate con un messaggio d'errore
105 client avviati simultaneamente (coda con limite 100)	esattamente 5 rifiuti, nessun errore non gestito
150 client avviati simultaneamente, ripetuto 3 volte	nessun errore non gestito, nessuna risorsa IPC residua in ciascuna esecuzione
Burst ripetuti fino a 4200 richieste consecutive (rilevamento dei worker terminati)	nessun evento spurio in tre ripetizioni indipendenti (12600 richieste), coda sempre vuota
admin status interrogato ripetutamente durante un carico concorrente, con la coda vicina alla saturazione	sempre riuscito dopo la correzione (in precedenza falliva sistematicamente)
Secondo server sulla stessa coda	rifiutato con messaggio
File inesistente, server non attivo	errore, codice di uscita 1
Server fermato con client in coda e in elaborazione, ripetuto 15 volte (143 interruzioni)	motivo specifico ricevuto nel 100% dei casi dopo la correzione (64% prima)
Ctrl-C su un client in coda	risorse rimosse
kill -9 su un client in coda	il server ripulisce entro circa 1 s
kill -9 su un client durante il trasferimento	il servizio riprende dopo il timeout di 10 s e serve la richiesta seguente
kill -9 sul server, prima che un client si connetta	la coda resta; al riavvio il messaggio indica come rimuoverla; il server successivo riusa e reinizializza il semaforo di presenza senza intervento manuale
Client connesso a una coda lasciata da un server ucciso con kill -9	rileva l'assenza del server tramite il semaforo di presenza ed esce con un messaggio chiaro in circa 2 s (prima: bloccato indefinitamente)
Client già in attesa quando il server viene ucciso con kill -9	rileva l'assenza entro circa 1 s (prima: bloccato indefinitamente)
Client in coda dietro due file da 2 MB, server sempre vivo	attesa di 8 s senza falsi allarmi, risultato corretto

Come ordine di grandezza, un file di 20 MB viene elaborato in circa 0,3 s senza ritardo artificiale; l'opzione `-d` è stata usata solo per rendere osservabili le code.

8 Limitazioni e possibili estensioni

- **Risorse residue in casi estremi.** Non sono recuperabili le risorse di un client ucciso tra la loro creazione e l'invio della richiesta, né la coda di un server ucciso con `kill -9` (segnalata al riavvio). Il controllo di esistenza dei client si basa sul pid, che in linea di principio potrebbe essere riassegnato a un altro processo.
- **Dimensioni fisse.** Il limite di processi worker configurabile è al più 64; le richieste in attesa sono al più 100 e quelle eccedenti sono rifiutate con un messaggio d'errore.
- **Portabilità.** Il sistema è stato collaudato solo su Linux. Per agevolare l'esecuzione su macOS la `union semun` non viene ridefinita su piattaforme Apple e i messaggi sono di dimensione ridotta, ma tale configurazione non è stata verificata. Le code System V di macOS hanno limiti di capacità predefiniti sensibilmente inferiori.
- **Istanza unica.** La chiave della coda è fissa: due istanze indipendenti del server (per esempio di utenti diversi) non possono coesistere.

9 Organizzazione del codice

File	Contenuto
CMakeLists.txt	configurazione della compilazione
src/server.c	ciclo del server, schedulazione, worker, comandi di amministrazione
src/client.c	creazione delle risorse, trasferimento del file, attesa della risposta, <code>semOpClient</code>
src/admin.c	comandi <code>limit</code> e <code>status</code>
src/semaphore.c	funzione <code>semOp</code> (P/V su un semaforo)
src/err_exit.c	funzione <code>errExit</code>
include/defines.h	costanti, tipi di messaggio, strutture condivise
include/semun.h	definizione della <code>union semun</code>
include/semaphore.h	dichiarazione di <code>semOp</code>
include/err_exit.h	dichiarazione di <code>errExit</code>
tests/run_tests.sh	suite di test automatica

Tabella 6: Organizzazione dei sorgenti.